Robot Documentation

Alexander Greus, Kenneth Thompson, Dylan Zemlin, Spencer Smith

October 29, 2025

1 Reactive Architecture

1.1 Chosen Architecture

We kept the same general architecture (Subsumption) as project 1 except removing most reactive layers Each layer is placed into a list, with earlier/higher layer able to override and inform lower/later layers. Again, this architecture was a good choice, since we can separate all the required behaviors/tasks into individual layers. As well, in this project higher level behaviors inform the decisions of lower level behaviors, so we again can mirror that with our priority list data structure. As well, we are glad to have chosen this architecture in project 1, as we were able to simply add and adjust behaviors very easily for project 2. Though we do not implement every layer as described in the Subsumption paper, we do implement something equivalent to the 'plan changes', 'monitor changes', and the 'avoid objects' layers.

1.2 Architecture Code - High Level Overview

The implementation of subsumption can be found in a few different places

- **Behavior Priority List** In the main loop, a list of behaviors is checked in the order inserted, that is higher priority behaviors are earlier in the list. The first behavior that changes any type of robot state, e.g. velocity, heading, plan state, takes control of that loop iteration and will prevent any further behaviors from triggering. For example the NavigatorBehavior ensures the robot is making progress towards its currently desired waypoint by submitting velocities and headings, and if does so lower behaviors do not run for that iteration.

- **Object Oriented Behaviors:** Each behavior is represented as a class (TaskManager-Behavior, NavigatorBehavior, PathFollowerBehavior), inheriting from a base Behavior class. This design makes it incredibly simple to implement the chosen architecture as we can easily put a list of behaviors together that can be ran using the same virtual function (behavior-¿run()).

- **Sensor Data:** Each behavior has access to a context object that contains the latest sensor reading and global state information such as whether or not the robot is escaping, etc.

1.3 Data Structures

1.3.1 Bot

The bot is our main data structure, which is just a class, with members including the Context, which as discussed before has sensor data and any other pieces of shared state, any required publishers and subscribers, as well as our priority list of behaviors and pending tasks. The behavior priority list and pending tasks are simply arrays of their respective element types, that being objects inheriting from the Behavior base class.

2 Algorithms and Behaviors

The behaviors as listed below are in order from highest priority to lowest priority (e.g. bumper is highest).

2.1 Bumper Behavior

The bumper sensor detects direct collisions with obstacles. If activated, this behavior issues an immediate stop command and disables teleoperation.

2.2 Escape Behavior

When the robot is blocked both left and right, it performs an escape routine that is implemented by selecting a random turn angle (using uniform sampling as per the instructions) to rotate approximately 180 degrees away from the obstacle. The escape behavior remains active until the robot completes its turn.

2.3 Avoidance Behavior

If an obstacle is detected on one side or directly in front, the avoidance algorithm determines an angular velocity to steer the robot away. This is done by comparing distances measured by the laser scanner on the left and right sides, then generating a turn speed.

2.4 Random Turn Behavior

To prevent the robot from becoming stuck in repetitive left/right movements the random turn behavior occasionally interrupts forward motion. After traveling a certain distance a random small adjustment which prevents the robot from getting stuck.

2.5 Forward Behavior

When no other behavior is triggered the robot defaults to forward motion.